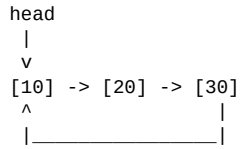
Circular Linked List in Java

Separate DSA notes • Circular singly and doubly linked lists, traversal safety, operations, and problems

1. What is a Circular Linked List?

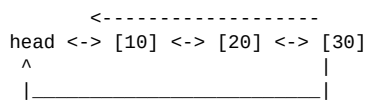
In a normal linked list, the final node points to null. In a **circular linked list**, the final node points back to an earlier node—usually the head—so traversal can continue in a loop.

Circular singly linked list (CSLL)



Circular doubly linked list (CDLL)

Each node has prev and next references. The last node's next points to head, and head.prev points to the last node. This supports circular traversal in both directions.



2. Where is circular linking useful?

- Round-robin CPU scheduling: after one process gets a time slice, scheduling can continue to the next process.
- Turn-based games: next player's turn follows the current player.
- Repeating playlists or carousel navigation.
- Circular buffers and cyclic task rotation (although a circular linked list is not the only implementation).

3. Node structure and traversal

```
class CNode {
    int data;
    CNode next;

    CNode(int data) {
        this.data = data;
    }
}
```

A standard while (current != null) traversal is wrong for a non-empty circular list because current never becomes null. Instead, process at least one node and stop when you return to the starting node.

```
static void display(CNode head) {
    if (head == null) {
        System.out.println("empty");
        return;
    }

    CNode current = head;
    do {
        System.out.print(current.data + " -> ");
        current = current.next;
    } while (current != head);

    System.out.println("(back to head)");
}
```

Why do-while? A non-empty list must print its head at least once. The empty-list check prevents dereferencing null.

4. Insert into a circular singly linked list

For a simple head-only implementation, inserting at the end requires finding the node whose next is head. This takes O(n). Keeping a tail reference can make insertion at the end O(1).

```
static CNode addFirst(CNode head, int value) {
    CNode node = new CNode(value);
```

```
        if (head == null) {
            node.next = node;       // points to itself
            return node;
        }

        CNode tail = head;
        while (tail.next != head) {
            tail = tail.next;
        }

        node.next = head;
        tail.next = node;
        return node;                // new head
    }
```

5. Insert at end and delete head

```
static CNode addLast(CNode head, int value) {
    CNode node = new CNode(value);

    if (head == null) {
        node.next = node;
        return node;
    }

    CNode tail = head;
    while (tail.next != head) {
        tail = tail.next;
    }

    tail.next = node;
    node.next = head;
    return head;
}

static CNode removeFirst(CNode head) {
    if (head == null) return null;

    // Only one node
    if (head.next == head) return null;

    CNode tail = head;
    while (tail.next != head) {
        tail = tail.next;
    }

    head = head.next;
    tail.next = head;
    return head;
}
```

Trace the one-node case carefully: when a node points to itself, deleting it must return null. Otherwise, the code may leave a one-node loop behind.

6. Search in a circular list

```
static boolean contains(CNode head, int target) {
    if (head == null) return false;

    CNode current = head;
    do {
        if (current.data == target) return true;
        current = current.next;
    } while (current != head);

    return false;
}
```

Time complexity is O(n); extra space is O(1). The loop stops after one complete circuit, even when the value is absent.

7. Circular singly vs circular doubly

Aspect	Circular singly	Circular doubly
Node links	next	prev and next
Movement	Forward around the circle	Forward and backward
Memory	Lower	Higher
Endpoint links	tail.next = head	tail.next = head and head.prev = tail
Main risk	Infinite loop if stop condition is wrong	Broken forward/backward consistency

8. Worked problem: Josephus-style elimination

Imagine n people arranged in a circle. Starting from person 1, count k people and remove the kth; continue until one remains. The circular structure models the repeating order naturally.

A simple list simulation can remove the element at an index repeatedly. This version is easy to understand, though array-list removal can make it less efficient for large inputs.

```
import java.util.ArrayList;
import java.util.List;

static int josephus(int n, int k) {
    List<Integer> people = new ArrayList<>();
    for (int i = 1; i <= n; i++) {
        people.add(i);
    }

    int index = 0;
    while (people.size() > 1) {
        index = (index + k - 1) % people.size();
        people.remove(index);
    }
    return people.get(0);
}

// Example: josephus(5, 2) returns 3
```

Index calculation: k - 1 advances to the kth person because the current index is counted as position one. Modulo wraps the index around the circle.

9. Common bugs and debugging checklist

• Using current != null as the only stop condition in a circular list.
• Forgetting the self-loop when creating the first node.
• Failing to reconnect tail.next to the updated head after deleting the head.
• Forgetting the single-node case.
• Accidentally creating two separate cycles or breaking the cycle while inserting.
• Printing forever because the stop condition checks the wrong node.

10. Complexity and practice

Operation	Head-only CSLL	With tail reference
Traverse / search	O(n)	O(n)
Insert at beginning	O(n) if tail must be found	O(1)
Insert at end	O(n)	O(1)
Delete head	O(n) if tail must be found	O(1)

Practice: (1) count nodes in one circuit, (2) insert after a given value, (3) delete a target value, (4) implement a circular doubly linked list, (5) simulate round-robin turns.

Interview Aspect: State the invariant clearly: after one full traversal, the current reference returns to the starting node. Explain how empty and single-node cases differ, and how maintaining a tail reference changes insertion complexity.